最終プロジェクト：推し文豪ボット

自分のGPTを育てる — そして文豪テストの種明かし

rkskmt

文豪テスト、開封

初回の4つの文章。投票の結果を覚えているだろうか。

	正体
A（喜多床へ行って顔さえ剃って貰やあ…）	**本物の漱石**
B（元来主人のように関する主人は…）	**機械**
C（迷亭はとぼけた顔をして…）	**本物の漱石**
D（まあどうですか、全く君の事じゃ…）	**機械**

- B と D を書いたのは：**82万パラメータ・『吾輩は猫である』1冊・3,500回訓練**の文字レベルGPT
- つまり、君たちが前回まで動かしてきた **`ckpt_best.pt` そのもの**（temperature 0.8・top-k 40）
- 初回の「怪しい箇所」の採点表を取ってある人は、ここで答え合わせを — 君の目は機械のしっぽを掴んでいたか

種明かしの種明かし

- 生成文と本文の一致は最長でも**9文字** — 暗記の吐き出しではなく、**文体の統計**が書いた偽物
- よく読めば文法は壊れている。それでも投票が割れるのは、**「漱石の音」が確かに写っている**から
- 32万字＋50分の訓練でこれなら、**推し作家**ならどうなる？ — というのが今日の全部

⚠ 最終プロジェクト

青空文庫から好きな作家・作品を選び、自分のGPTを育て、**偽物の一文**を提出せよ。 最終日は全員の偽物＋本物でクラス対抗・文豪テスト大会を開く。

3段階で仕上げる

段階	見せるもの	完了の目印
材料検査	作家・作品名、字数、語彙サイズ、冒頭200字	洗った本文が読める
試運転	500回までの train / val、所要時間	**`my_ckpt_500.pt`** ができる
本番	選んだ重み、偽物と本物、生成のツマミ	選択理由を説明できる

- 途中で失敗しても、最後に成功した段階から再開する
- 最終日に初めて訓練するのではなく、段階ごとに動作を確かめる

材料選び — 青空文庫

- **青空文庫** は著作権の切れた作品の宝庫（漱石・太宰・芥川・賢治・乱歩……）
- 作品の **図書カード** から「テキストファイル（ルビあり）」の zip をダウンロード → 解凍して **`.txt`** を **`data/`** に置く

> **⚠ 材料の量が成績を決める**
>
> - **10万字以上** が快適圏（長編1本、または短編の詰め合わせ）
> - 短い作品は速く仕上がるが、**すぐ過学習する**（後述の実測を見よ）
> - 短編派は同じ作家の複数作品を1ファイルに **連結** するのがおすすめ

下ごしらえ — ルビと注記を洗う

- 青空文庫のテキストは Shift_JIS で、ルビ《…》や注記［＃…］が混ざっている。洗ってから使う

Code

```
import re

raw = open("data/ginga_tetsudo.txt", encoding="cp932").read()   # DLした生テキスト

parts = re.split(r"-{10,}\r?\n", raw)       # 冒頭の説明書きを外す
body = parts[2]
body = re.split(r"\r?\n底本：", body)[0]     # 末尾の書誌情報を外す
body = body.replace("｜", "")
body = re.sub(r"《[^》]*》", "", body)       # ルビ
body = re.sub(r"［＃[^］]*］", "", body)     # 注記
body = body.replace("\r\n", "\n").strip() + "\n"

open("data/mybungo.txt", "w", encoding="utf-8").write(body)
print(len(body), "字 / 語彙", len(set(body)))
print(body[:200])
```

- 複数作品を使う場合は、この処理を作品ごとに行って **body** を足し合わせる
- **最初に「字数と語彙」を必ず表示する** — 材料の下見をしない料理人にならない

育てる — 道具はすべて手元にある

- 訓練コードは**小さなGPTの回のものを1行だけ差し替える**：読み込むファイル名だけ
- **gpt.py** を自分のスクリプトと同じフォルダに置く（小さなGPTの回と同じ配置）

```python
import time

import torch
import torch.nn.functional as F
from gpt import GPT

torch.set_num_threads(1)
text = open("data/mybungo.txt", encoding="utf-8").read()    # ← ここだけ差し替え
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
data = torch.tensor([stoi[c] for c in text])
n = int(len(data) * 0.9)
train_data, val_data = data[:n], data[n:]

def get_batch(d):
    ix = torch.randint(len(d) - 65, (32,))
    x = torch.stack([d[i:i + 64] for i in ix])
    y = torch.stack([d[i + 1:i + 65] for i in ix])
    return x, y

torch.manual_seed(42)
model = GPT(vocab_size=len(chars), n_embd=96, n_head=4, n_layer=2, block_size=64)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3)

MAX_ITERS = 1000
started = time.perf_counter()

for it in range(1, MAX_ITERS + 1):
    xb, yb = get_batch(train_data)
    opt.zero_grad()
    loss = F.cross_entropy(model(xb).view(-1, len(chars)), yb.view(-1))
    loss.backward()
    opt.step()
    if it == 100:
        sec_per_iter = (time.perf_counter() - started) / 100
        print(f"このPCの見込み: 500回 {sec_per_iter * 500 / 60:.1f}分 / "
              f"1000回 {sec_per_iter * 1000 / 60:.1f}分")
    if it % 250 == 0:
        with torch.no_grad():
            xv, yv = get_batch(val_data)
            vl = F.cross_entropy(model(xv).view(-1, len(chars)), yv.view(-1))
        print(f"{it:5d}回 | train {loss.item():.3f} | val {vl.item():.3f}")
        torch.save({"cfg": dict(n_embd=96, n_head=4, n_layer=2, block_size=64),
                    "vocab": "".join(chars),
                    "state_dict": model.state_dict()}, f"data/my_ckpt_{it}.pt")
```

- 250回ごとに val を表示し、**重みを保存**していく — 早期終了は「あとから底を選ぶ」で実現できる

15〜30分に収める

- 標準は **`MAX_ITERS = 1000`**。まず100回の表示で、**自分のPCの所要時間**を確認
- 1000回の見込みが30分を超えるなら **`750`**、15分未満なら **`1500`** まで延長してよい
- **500回の重みができれば試運転は完了** — 時間内に終わらなくても、そこから生成と分析へ進める
- 速さを競う課題ではない。回数より、train / val を見て重みを選んだ理由が採点対象

何回回すべきか — 実測の目安

同じ設定で材料の量だけ変えた実測：

材料	字数 / 語彙	val の底	そのときの様子
『吾輩は猫である』	32万 / 3,045	**2,500〜3,500回**（約20分）	文体が出る
『銀河鉄道の夜』	3.9万 / 1,050	**約1,000回**（数分）	それ以降は val が悪化

- **小さい材料ほど、早く仕上がり、早く腐る**（過学習）— val の数字だけが頼り
- 語彙が小さいと1回あたりの計算も軽い。**まず500回**回して、val の動きを見てから延長を決める

> **ヒント**
>
> 損失の数字はデータが違うと直接比べられない（語彙数が違えば「当てずっぽう」の損失から違う）。 比べてよいのは**同じデータでの train と val** — この2本の開きだけが過学習の証拠。

偽物を提出する

```python
import torch
import torch.nn.functional as F
from gpt import GPT

BEST_ITER = 500   # 自分のログで val が最小だった回へ変更
blob = torch.load(f"data/my_ckpt_{BEST_ITER}.pt", weights_only=True)
model = GPT(vocab_size=len(blob["vocab"]), **blob["cfg"])
model.load_state_dict(blob["state_dict"])
model.eval()
chars = blob["vocab"]
stoi = {c: i for i, c in enumerate(chars)}

torch.manual_seed(7)
idx = torch.tensor([[stoi[c] for c in "その夜"]])
out = model.generate(idx, 120, temperature=0.8, top_k=40)
print("".join(chars[i] for i in out[0].tolist()))
```

本番提出（文豪テスト大会のエントリーシート）：

0. 作家名・使った作品と字数・語彙サイズ
1. **偽物1本（120字前後）**と、生成に使ったツマミ（temperature / top-k / シード / 書き出し）
2. 同じ作家の**本物1本**（偽物と長さを揃える）
3. train / val 損失の推移（printの写しでよい）と、**どの回の重みを選んだか＋その理由**

提出前の身体検査 — 盗作チェック

- 過学習ぎみの重みや低い temperature では、**本文の丸写しに近い文**が出ることがある。偽物は文体の**創作**であって暗記の吐き出しではないこと — 大会のフェアプレー
- コーパスとのひと続きの**最長一致**を測ってから提出する：

```python
def longest_match(fake, text):
    best = ""
    for i in range(len(fake)):
        j = i + len(best) + 1
        while j <= len(fake) and fake[i:j] in text:
            best = fake[i:j]
            j += 1
    return best

fake = "……"   # 提出する偽物をここに
text = open("data/mybungo.txt", encoding="utf-8").read()
m = longest_match(fake, text)
print(len(m), "字:", m)
```

- 参考値：文豪テストの機械文 B・D は最長一致 **8字・9字**（決まり文句レベル）
- 逆に**本物の文**を入れれば全文一致するはず — 検査コード自体の動作確認になる
- 文が丸ごと一致したら：val の底より**手前**の重みを選び直すか、temperature を少し上げる

大会のルール

- 出題者以外は「本物・偽物」を投票 → 騙した票数がスコア
- 発表者は最後に**ツマミと訓練の記録**を開示（再現できない魔法は禁止）
- 審査観点はふたつ：**どれだけ騙せたか**と、**なぜ騙せたかを説明できるか**

> ⓘ **ノート**
>
> 騙せなかった偽物にも価値がある — **どこでバレたか**（文法？語彙？リズム？）を言語化できれば、 それはモデルの弱点分析であり、立派な技術報告になる。

大会の進行

1組あたり3分弱。流れ：

0. 出題：**本物と偽物を並べて**提示（どちらが偽物かは伏せる。並び順はコイントスで）
1. 全員30秒の黙読 → 「偽物はどっち？」で一斉に挙手
2. 開票 → 出題者が正解と**ツマミ・訓練の記録**を開示（1分）
3. 記録係が「騙した人数」をスコア表に記入
   - 全組が終わったら表彰は2部門：**最多得票の偽物**と、**最も鮮やかにバレた偽物**（バレた理由の解剖が見事だったもの）
   - 時間が余ったら：優勝作をみんなで解剖 — どの一節が「らしさ」を出したのか

講義全体のまとめ — 部品総覧

部品	正体	作った回
トークン化・語彙	文字に番号を振る	文字を数にする
確率で次を選ぶ	数える→サイコロ	数えるだけの言語モデル
交差エントロピー学習	数えの一般化	学習で当てる
`nn.Embedding`	表引き＝ことばの地図	ことばの地図
過学習との再会	窓を広げすぎた罰	窓を広げる
Attention	内積→softmax→重み付き平均	注目という発想 / Q・K・V
位置埋め込み・因果マスク	順序を教え、未来を封じる	順序とカンニング
マルチヘッド・FFN	視線の複線化＋考える場所	束ねて、考える
残差・LayerNorm	勾配の迂回路＋入力揃え	深く積む
訓練と早期終了	val の底で止める	小さなGPTを訓練する
temperature・top-k	生成のツマミ	生成を操る

> ⓘ **約束の回収**
>
> **ブラックボックス、ゼロ。** 文豪テストの偽物は、この表の部品だけで書かれた。

完成した設計図　作家の本文 → トークン化 → GPT訓練 → val最良の重み → **文体をまねた生成文**

この先の世界

- **語彙を賢くしたら？** — 文字をやめてサブワードへ：**BPE**（回数外の参考資料で自作できる）
- **もっと大きくしたら？** — 同じ設計図のまま21万倍が ChatGPT の土台（前回）。スケールの先は現在進行形の研究
- **文章以外は？** — 画像も音声も「トークン列にして次を当てる」の親戚が動いている。開け方は知ってのとおり：**forward は何か・損失は何か・勾配はどう流れるか**

> ⓘ **持ち帰り**
>
> 家族か友人に文豪テストを出題してみること。騙せたら、その偽物は君の部品が書いた文章だ。 騙せなくても、**なぜバレたかを説明できる** — それがこの講義の到達点。